

Practical-Week5

1. Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>
#include <string.h>

#define BUFFER_SIZE 100

int main() {
    int pipefd[2];
    pid_t pid;

    char buffer[BUFFER_SIZE];
    int data[] = {10, 20, 30, 40, 50};
    int n = 5;

    struct timespec start, end;

    // Create anonymous pipe
    if (pipe(pipefd) == -1) {
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    // Start measuring time
    clock_gettime(CLOCK_MONOTONIC, &start);

    // Create child process
    pid = fork();

    if (pid < 0) {
        perror("fork");
        exit(EXIT_FAILURE);
    }
```

```
ajithmutyala@Ajith:~$ nano producer_consumer.c
ajithmutyala@Ajith:~$ gcc producer_consumer.c -o producer_consumer
ajithmutyala@Ajith:~$ ./producer_consumer
Producer (Parent): Sending data...
Produced: 10
Produced: 20
Produced: 30
Produced: 40
Produced: 50

Consumer (Child): Receiving data...
Consumed: 10
Consumed: 20
Consumed: 30
Consumed: 40
Consumed: 50

Communication completed.
Communication time: 0.000368778 seconds
Data transferred: 20 bytes
Communication efficiency (throughput): 54233.17 bytes/sec
ajithmutyala@Ajith:~$ |
```

```

ajithmutyala@Ajith:~$ strace ./producer_consumer
execve("./producer_consumer", ["/producer_consumer"], 0x7ffc6e73ff80 /* 26
vars */) = 0
brk(NULL)                                     = 0x5d7638aed000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0
x791a5b414000
access("/etc/ld.so.preload", R_OK)           = -1 ENOENT (No such file or directo
ry)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=20943, ...}) = 0
mmap(NULL, 20943, PROT_READ, MAP_PRIVATE, 3, 0) = 0x791a5b40e000
close(3)                                     = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC)
= 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0 \250\2\0\0\0\0\0"...
, 832) = 832
pread64(3, "\6\0\0\0\4\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0"..
., 840, 64) = 840
fstat(3, {st_mode=S_IFREG|0755, st_size=2186512, ...}) = 0
pread64(3, "\6\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0"..
., 840, 64) = 840
mmap(NULL, 2231696, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x791a5b00
0000
mmap(0x791a5b028000, 1671168, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP
_DENYWRITE, 3, 0x28000) = 0x791a5b028000
mmap(0x791a5b1c0000, 319488, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0x1c0000) = 0x791a5b1c0000
mmap(0x791a5b20e000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_
DENYWRITE, 3, 0x20d000) = 0x791a5b20e000
mmap(0x791a5b214000, 52624, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_
ANONYMOUS, -1, 0) = 0x791a5b214000
close(3)                                     = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x791a5b40b000
arch_prctl(ARCH_SET_FS, 0x791a5b40b740) = 0
set_tid_address(0x791a5b40bd68)              = 580
set_robust_list(0x791a5b40ba20, 24)          = 0
rseq(0x791a5b40b6a0, 0x20, 0, 0x53053053) = 0
mprotect(0x791a5b20e000, 16384, PROT_READ) = 0
mprotect(0x5d7626947000, 4096, PROT_READ) = 0
mprotect(0x791a5b459000, 8192, PROT_READ) = 0

```

```

ajithmutyala@Ajith:~$ strace -f -e trace=process,ipc,read,write,close,wait4
./producer_consumer
execve("./producer_consumer", ["/producer_consumer"], 0x7ffdad6d2098 /* 26
vars */) = 0
close(3) = 0
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\3\0>\0\1\0\0\0 \250\2\0\0\0\0"...
, 832) = 832
close(3) = 0
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARPID|CLONE_CHILD_SETTID|SIGCHLD
)stage: Process 604 attached
[pid 603] child_tidptr=0x7309ab5fca10) = 604
[pid 603] close(3) = 0
[pid 604] close(4) = 0
[pid 603] write(1, "Producer (Parent): Sending data."..., 35Producer (Pare
nt): Sending data...
) = 35
[pid 604] write(1, "\n", 1 <unfinished ...>
[pid 603] write(4, "\n\0\0\0", 4
<unfinished ...>
[pid 604] <... write resumed>) = 1
[pid 603] <... write resumed>) = 4
[pid 604] write(1, "Consumer (Child): Receiving data"..., 36Consumer (Chil
d): Receiving data...
<unfinished ...>
[pid 603] write(1, "Produced: 10\n", 13Produced: 10
<unfinished ...>
[pid 604] <... write resumed>) = 36
[pid 603] <... write resumed>) = 13
[pid 604] read(3 <unfinished ...>
[pid 603] write(4, "\24\0\0\0", 4 <unfinished ...>
[pid 604] <... read resumed> "\n\0\0\0", 4) = 4
[pid 603] <... write resumed>) = 4
[pid 603] write(1, "Produced: 20\n", 13 <unfinished ...>
[pid 604] write(1, "Consumed: 10\n", 13Produced: 20
<unfinished ...>
[pid 603] <... write resumed>) = 13
Consumed: 10
[pid 604] write(4, "\36\0\0\0", 4 <unfinished ...>
[pid 604] <... write resumed>) = 13
[pid 603] <... write resumed>) = 4
[pid 604] read(3 <unfinished ...>

```

Report: The strace command is used to trace system calls made by the producer and consumer processes, such as read(), write(), close(), and wait(). The output shows that the parent produced 10, 20, 30, 40, 50 and the child consumed the same data successfully, with a communication time of about 0.00037 seconds and throughput of 54233.17 bytes/sec.

2. Develop a program that executes the equivalent of the shell command: `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls. 01:32 PM

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pipefd[2];
    pid_t pid1, pid2;

    if (pipe(pipefd) == -1) {
        perror("pipe");
        exit(1);
    }

    // First child: executes ls -l
    pid1 = fork();

    if (pid1 == -1) {
        perror("fork");
        exit(1);
    }

    if (pid1 == 0) {
        // Redirect stdout to pipe
        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[0]);
        close(pipefd[1]);

        execlp("ls", "ls", "-l", NULL);

        perror("execlp ls");
        exit(1);
    }

    // Second child: executes grep ".c"
    pid2 = fork();
```

```

ajithmutyala@Ajith:~$ nano ls_grep.c
ajithmutyala@Ajith:~$ gcc ls_grep.c -o ls_grep
ajithmutyala@Ajith:~$ ./ls_grep
-rw-r--r-- 1 ajithmutyala ajithmutyala 862 Aug 20 05:44 builtin.c
-rw-r--r-- 1 ajithmutyala ajithmutyala 861 Aug 3 06:09 command_executor.c
-rwxr-xr-x 1 ajithmutyala ajithmutyala 16216 Aug 14 10:05 filecopy
-rw-r--r-- 1 ajithmutyala ajithmutyala 728 Aug 14 10:04 filecopy.c
-rw-r--r-- 1 ajithmutyala ajithmutyala 1148 Aug 29 09:46 ls_grep.c
-rw-r--r-- 1 ajithmutyala ajithmutyala 813 Aug 20 05:28 no_zombie.c
-rwxr-xr-x 1 ajithmutyala ajithmutyala 16344 Aug 20 05:33 path_search
-rw-r--r-- 1 ajithmutyala ajithmutyala 1714 Aug 20 05:32 path_search.c
-rwxr-xr-x 1 ajithmutyala ajithmutyala 16392 Aug 20 04:50 process
-rw-r--r-- 1 ajithmutyala ajithmutyala 1821 Aug 20 04:55 process.c
-rw-r--r-- 1 ajithmutyala ajithmutyala 1803 Aug 20 04:59 process.c.save
-rwxr-xr-x 1 ajithmutyala ajithmutyala 16217 Aug 14 13:23 process_state
-rw-r--r-- 1 ajithmutyala ajithmutyala 1009 Aug 14 13:28 process_state.c
-rwxr-xr-x 1 ajithmutyala ajithmutyala 16440 Aug 29 08:05 producer_consumer
-rw-r--r-- 1 ajithmutyala ajithmutyala 1985 Aug 29 08:05 producer_consumer.c
drwxr-xr-x 3 ajithmutyala ajithmutyala 4096 Aug 13 16:52 shell_project
-rw-r--r-- 1 ajithmutyala ajithmutyala 40 Aug 14 10:05 source.txt
-rw-r--r-- 1 ajithmutyala ajithmutyala 68 Aug 29 08:12 textfile.c
-rw-r--r-- 1 ajithmutyala ajithmutyala 11448 Aug 14 13:08 trace.txt
-rw-r--r-- 1 ajithmutyala ajithmutyala 1304 Aug 20 05:38 variable_expand.c
-rw-r--r-- 1 ajithmutyala ajithmutyala 1057 Aug 20 05:29 waitpid_demo.c
-rw-r--r-- 1 ajithmutyala ajithmutyala 731 Aug 20 05:12 zombie.c
ajithmutyala@Ajith:~$ |

```

```

ajithmutyala@Ajith:~$ strace -f -e trace=process,ipc,dup2,execve,close,wait4 ./ls_grep
execve("./ls_grep", ["/ls_grep"], 0x7ffc558432b8 /* 26 vars */) = 0
close(3) = 0
close(3) = 0
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD, child_tidptr=0x7b338ec9ea10) = 1498
strace: Process 1498 attached
[pid 1497] clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD, child_tidptr=0x7b338ec9ea10) = 1499
[pid 1498] dup2(4, 1) = 1
strace: Process 1499 attached
[pid 1497] <... clone resumed>, child_tidptr=0x7b338ec9ea10) = 1499
[pid 1498] close(3) = 0
[pid 1498] close(4) = 0
[pid 1498] close(3) = 0
[pid 1498] execve("/usr/local/sbin/ls", ["ls", "-l"], 0x7ffdebfd098 /* 26 vars */) <unfinished ...>
[pid 1497] close(4 <unfinished ...>)
[pid 1497] dup2(3, 0 <unfinished ...>)
[pid 1497] <... close resumed> = 0
[pid 1497] <... execve resumed> = -1 ENOENT (No such file or directory)
[pid 1497] wait4(1498 <unfinished ...>)
[pid 1499] <... dup2 resumed> = 0
[pid 1498] execve("/usr/local/bin/ls", ["ls", "-l"], 0x7ffdebfd098 /* 26 vars */) <unfinished ...>
[pid 1499] close(3 <unfinished ...>)
[pid 1499] <... execve resumed> = -1 ENOENT (No such file or directory)
[pid 1499] <... close resumed> = 0
[pid 1499] execve("/usr/sbin/ls", ["ls", "-l"], 0x7ffdebfd098 /* 26 vars */) <unfinished ...>
[pid 1499] close(4 <unfinished ...>)
[pid 1499] <... execve resumed> = -1 ENOENT (No such file or directory)
[pid 1499] <... close resumed> = 0
[pid 1498] execve("/usr/bin/ls", ["ls", "-l"], 0x7ffdebfd098 /* 26 vars */) <unfinished ...>
[pid 1499] execve("/usr/local/sbin/grep", ["grep", ".c"], 0x7ffdebfd098 /* 26 vars */) = -1 ENOENT (No such file or directory)
[pid 1499] execve("/usr/local/bin/grep", ["grep", ".c"], 0x7ffdebfd098 /* 26 vars */) = -1 ENOENT (No such file or directory)
[pid 1497] <... execve resumed> = 0
[pid 1499] execve("/usr/sbin/grep", ["grep", ".c"], 0x7ffdebfd098 /* 26 vars */) = -1 ENOENT (No such file or directory)
[pid 1499] execve("/usr/bin/grep", ["grep", ".c"], 0x7ffdebfd098 /* 26 vars */) = 0
[pid 1498] close(3) = 0
[pid 1499] close(3) = 0
[pid 1499] close(3 <unfinished ...>)
[pid 1499] close(3) = 0
[pid 1498] <... close resumed> = 0
[pid 1499] close(3) = 0
[pid 1499] close(3) = 0

```

Report: The program successfully implements `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls. It redirects the output of `ls -l` through the pipe and displays only the `.c` files using `grep`.